

COE180.1

Introduction to Hardware Description Language

Lab Manual (2025)

Name & Student number



Contents

2	Design of Binary Adders	2
2.1	Objectives	2
2.2	Introduction	2
2.3	Background Theory	4
2.4	SystemVerilog Implementation Examples: The Half Adder	5
2.5	Lab Tasks and Procedure	9
2.6	Challenge Problem: Full Adder-Subtractor Circuit	9
2.7	Pre-lab Questions	10
2.8	Post-lab Questions	10
2.9	Lab Report Requirements	11
2.10	Grading Rubric	11

2 Design of Binary Adders

2.1 Objectives

Upon successful completion of this lab, you will be able to:

- Understand the principles of fundamental combinational arithmetic circuits.
- Design and implement half adders and full adders in SystemVerilog.
- Design and implement half subtractors and full subtractors in SystemVerilog.
- Apply all three modeling styles (structural, dataflow, behavioral) to a practical design.
- Simulate and verify the functionality of these circuits using a testbench.
- Analyze the trade-offs between different modeling approaches.
- Understand the advantages of SystemVerilog over traditional Verilog.

2.2 Introduction

In digital electronics, arithmetic operations form the backbone of computation. The fundamental operations of addition and subtraction are realized through specialized combinational circuits known as adders and subtractors. This lab explores the design and implementation of these crucial circuits using SystemVerilog HDL, providing hands-on experience with different abstraction levels in digital design.

2.2.1 Transition to SystemVerilog

This lab introduces a significant update from Lab 1: we are now using **SystemVerilog** instead of traditional Verilog. SystemVerilog is a superset of Verilog that includes many enhancements for both design and verification. This transition represents industry best practices, as SystemVerilog has become the standard for modern digital design.

2.2.2 SystemVerilog vs. Traditional Verilog

SystemVerilog extends traditional Verilog with numerous features that improve code readability, reusability, and verification capabilities. While we can still use traditional Verilog syntax, adopting SystemVerilog practices will better prepare you for industry work.

Table 2.1: Key differences between Traditional Verilog and SystemVerilog

Feature	Traditional Verilog	SystemVerilog
Primary data types	<code>wire</code> , <code>reg</code>	<code>logic</code> (replaces both in most cases)
Combinational processes	<code>always @(*)</code>	<code>always_comb</code> (better checking)
Port declarations	Separate direction and type	Combined direction and type
Assignment operators	Blocking (<code>=</code>) and non-blocking (<code><=</code>)	Additional operators: <code>++</code> , <code>--</code> , <code>+=</code> , etc.
Package system	Limited support	Robust package support for code organization
Enumerated types	Not available	<code>enum</code> for finite state machines
User-defined types	Limited	<code>typedef</code> for custom types
Interface support	Module ports only	<code>interface</code> construct for bus modeling
Assertions	Limited	Powerful assertion system (SVA)
Randomization	Manual	Built-in randomization constraints

2.2.3 Adders

Adders are digital circuits that perform binary addition. The hierarchy begins with:

- **Half Adder:** Takes two single-bit inputs (A and B) and produces two outputs: sum (S) and carry-out (Cout). It is suitable for the least significant bit addition.
- **Full Adder:** Extends the half adder with a carry-in (Cin) input, enabling multi-bit addition by cascading multiple full adders.

2.2.4 Subtractors

Subtractors perform binary subtraction. Similar to adders, they have a basic hierarchy:

- **Half Subtractor:** Handles single-bit subtraction.
- **Full Subtractor:** Incorporates a borrow-in (Bin) input for multi-bit subtraction.

2.2.5 Practical Applications

The simple adders and subtractors you will build in this lab are the fundamental components of much larger and more complex systems. Their applications are widespread:

- **Arithmetic Logic Units (ALUs):** This is the most direct and important application. ALUs are the heart of Central Processing Units (CPUs), responsible for performing all arithmetic and logical calculations. The multi-bit adders within an ALU are constructed by cascading the full adders you will design in this lab.
- **Address Calculation:** Processors constantly calculate memory addresses to fetch instructions and data. These calculations often involve simple addition, which is handled by dedicated adder circuits.
- **Graphics Processing Units (GPUs):** GPUs perform billions of calculations per second to render complex 3D scenes. These calculations heavily rely on highly optimized adder and multiplier circuits.
- **Digital Signal Processing (DSP):** In applications like audio filtering or image processing, DSPs use arithmetic circuits to implement mathematical functions like convolutions and Fourier transforms.

2.2.6 SystemVerilog HDL Modeling Styles

As in Lab 1, you will use multiple abstraction levels to describe these circuits, now with enhanced SystemVerilog syntax:

- **Behavioral Modeling:** Describes circuit functionality using procedural statements (`always_comb`, `if-else`, `case`).
- **Dataflow Modeling:** Focuses on signal flow using continuous assignments (`assign`) and Boolean expressions.
- **Structural Modeling:** Composes circuits by interconnecting lower-level components, promoting hierarchy and reuse.

Notable Information

This lab builds directly upon the fundamentals from Lab 1. Focus on how complex logic (like addition) can be built from the basic gates you have already mastered. Pay close attention to how a full adder can be constructed from simpler half adders—this is a key lesson in hierarchical design. Also note the transition to SystemVerilog, which provides more expressive power and industry-relevant syntax.

2.3 Background Theory

2.3.1 The Half Adder

A half adder adds two single-bit binary numbers. It has two inputs, A and B, and two outputs, a Sum (S) and a Carry-out (Cout).

$$S = A \oplus B$$

$$C_{out} = A \cdot B$$

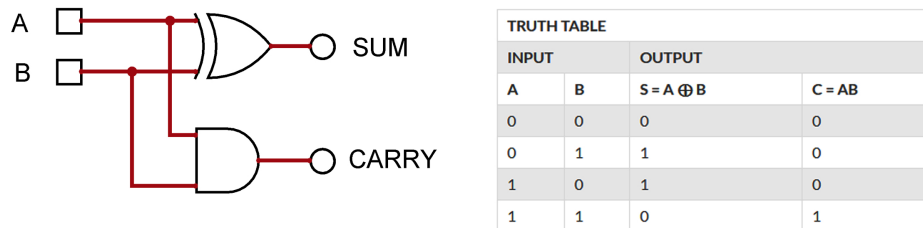


Figure 2.1: Half Adder circuit and truth table

2.3.2 The Full Adder

A full adder adds three single-bit binary numbers: A, B, and a Carry-in (Cin). This allows multiple full adders to be chained (or "cascaded") together to create multi-bit adders.

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

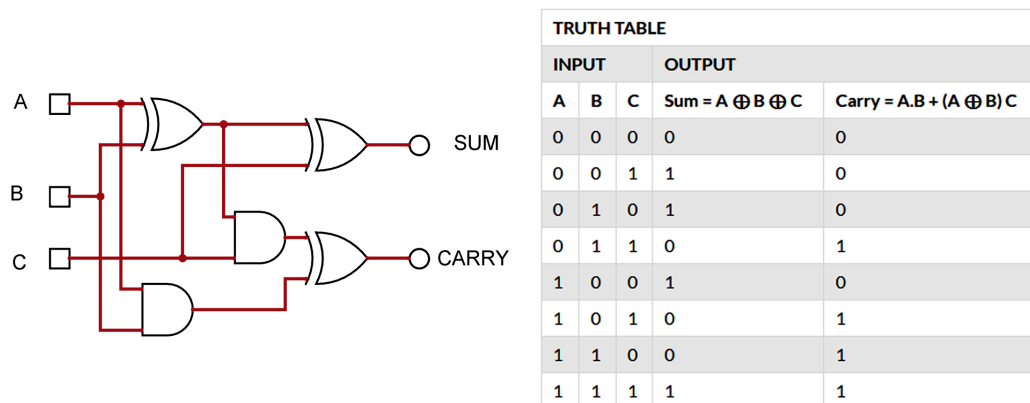


Figure 2.2: Full Adder circuit and truth table

2.3.3 Subtractors

Subtractors follow a similar pattern, producing a Difference (D) and a Borrow-out (Bout).

- **Half Subtractor:** $D = A \oplus B$ and $B_{out} = \bar{A} \cdot B$
- **Full Subtractor:** $D = A \oplus B \oplus B_{in}$ and $B_{out} = (\bar{A} \cdot B) + (B_{in} \cdot \overline{A \oplus B})$

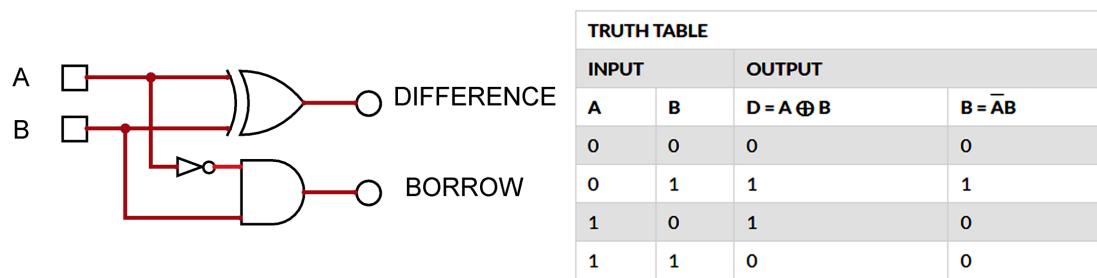


Figure 2.3: Half Subtractor circuit and truth table

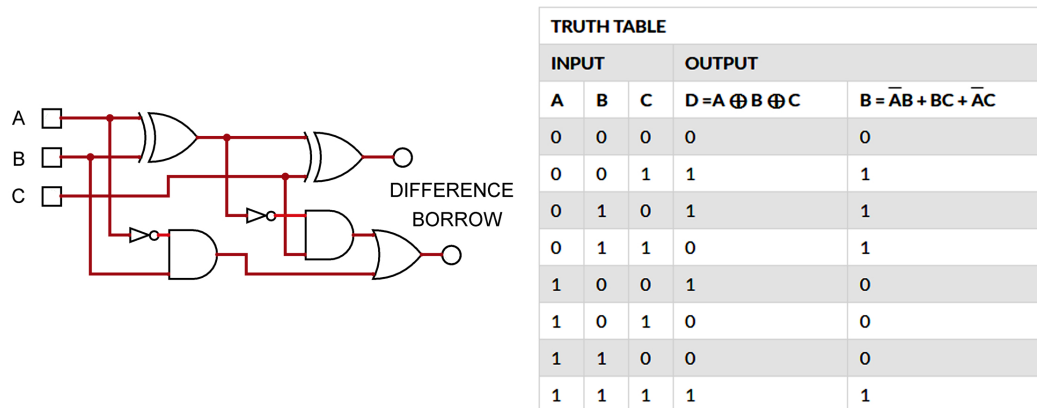


Figure 2.4: Full Subtractor circuit and truth table

2.4 SystemVerilog Implementation Examples: The Half Adder

This section provides side-by-side comparisons of implementing a half adder using traditional Verilog (from Lab 1) and modern SystemVerilog syntax across all three modeling styles. These examples highlight the syntactic improvements and enhanced capabilities of SystemVerilog.

2.4.1 SystemVerilog Advantages and Key Features

SystemVerilog introduces several significant improvements over traditional Verilog:

- **logic data type:** A unified data type that replaces both `reg` and `wire` in most cases, simplifying declarations and reducing errors.
- **always_comb procedural block:** Specifically designed for combinational logic, providing better compile-time checking than `always @()`.
- **Enhanced module port declarations:** Allows combined direction and type specification in port lists.
- **Named port connections:** Explicit `.port(signal)` syntax improves code readability and reduces connection errors.
- **Wildcard port connections:** The `.` operator automatically connects ports with matching names.
- **Stricter type checking:** Provides stronger typing than Verilog, helping catch errors earlier in the design process.
- **Enhanced procedural statements:** Additional loop constructs and assignment operators streamline testbench development.

2.4.2 Dataflow Modeling Comparison

Table 2.2: Dataflow Modeling: Traditional Verilog vs. SystemVerilog

Traditional Verilog	SystemVerilog
<pre> module half_adder_df (input a, b, output sum, carry); assign sum = a ^ b; assign carry = a & b; endmodule </pre>	<pre> module half_adder_df (input logic a, b, output logic sum, carry); assign sum = a ^ b; assign carry = a & b; endmodule </pre>

Hardware Inference

Both implementations directly infer two gates: an XOR gate for the **sum** and an AND gate for the **carry**. The SystemVerilog version uses the **logic** data type, which replaces both **reg** and **wire**, and declares port directions and types together for cleaner syntax.

2.4.3 Behavioral Modeling Comparison

Table 2.3: Behavioral Modeling: Traditional Verilog vs. SystemVerilog

Traditional Verilog	SystemVerilog
<pre> module half_adder_bh (input a, b, output reg sum, carry); always @(a or b) begin sum = a ^ b; carry = a & b; end endmodule </pre>	<pre> module half_adder_bh (input logic a, b, output logic sum, carry); always_comb begin sum = a ^ b; carry = a & b; end endmodule </pre>

Hardware Inference

The SystemVerilog implementation uses **always_comb** instead of **always @(*)**, which provides better compile-time checking for combinational logic. The **logic** data type eliminates the need for separate **reg** declarations, and the combined port declaration makes the code more concise.

2.4.4 Structural Modeling Comparison

Table 2.4: Structural Modeling: Traditional Verilog vs. SystemVerilog

Traditional Verilog	SystemVerilog
<pre> module half_adder_st (input a, b, output sum, carry); // Using gate primitives xor x1(sum, a, b); and a1(carry, a, b); endmodule </pre>	<pre> module half_adder_st (input logic a, b, output logic sum, carry); // Using gate primitives with // explicit port connections xor sum_gate (.a(a), .b(b), .z(sum)); and carry_gate (.a(a), .b(b), .z(carry)); endmodule </pre>

Hardware Inference

The SystemVerilog version uses named port connections (`.a(a)`) instead of positional connections, which improves code readability and reduces the risk of connection errors. The `logic` data type and combined port declarations are also utilized for consistency with other modeling styles.

2.4.5 Testbench Comparison

Table 2.5: Testbench Design: Traditional Verilog vs. SystemVerilog

Traditional Verilog	SystemVerilog
<pre> `timescale 1ns / 1ps module half_adder_tb; reg a, b; wire sum, carry; half_adder_df dut(a, b, sum, carry); initial begin \$dumpfile("half_adder_tb.vcd"); \$dumpvars(0, half_adder_tb); \$display("Testing Half Adder"); \$monitor("Time=%0t, a=%b, b=%b, sum=%b, carry=%b", \$time, a, b, sum, carry); a = 0; b = 0; #10; a = 0; b = 1; #10; a = 1; b = 0; #10; a = 1; b = 1; #10; \$finish; end endmodule </pre>	<pre> `timescale 1ns / 1ps module half_adder_tb; logic a, b; logic sum, carry; // Wildcard connection half_adder_df dut(.*); initial begin \$dumpfile("half_adder_tb.vcd"); \$dumpvars(0, half_adder_tb); \$display("Testing Half Adder"); \$monitor("Time=%0t, a=%b, b=%b, sum=%b, carry=%b", \$time, a, b, sum, carry); // Test using a loop for (int i = 0; i < 4; i++) begin {a, b} = i; #10; end \$finish; end endmodule </pre>

Hardware Inference

The SystemVerilog testbench demonstrates several improvements: (1) the `logic` data type replaces `reg` and `wire`, (2) wildcard connection (`.*`) automatically connects ports with matching names, (3) a `for` loop with an integer variable streamlines test vector generation, and (4) concatenation `a, b` allows compact assignment of multiple signals.

2.4.6 Simulation and Verification

1. Compile and Simulate:

```
iverilog -g2012 -o simulation half_adder_tb.v half_adder_df.v
vvp simulation
```

The -g2012 Compiler Flag: The Icarus Verilog command now includes the `-g2012` flag. This flag enables **SystemVerilog-2012 features** in the compiler. Without this flag, Icarus Verilog would default to traditional Verilog-2001 syntax, which would cause compilation errors for the SystemVerilog constructs we're using.

Notable Information

Backward Compatibility Note

While this lab emphasizes SystemVerilog syntax, traditional Verilog code remains fully compatible and will compile correctly with the `-g2012` flag. This backward compatibility allows for gradual adoption of SystemVerilog features and ensures that code from previous labs continues to work correctly.

2. View Waveforms:

```
gtkwave half_adder_tb.vcd
```

3. GTKWave Analysis:

- Navigate the hierarchy in the SST pane
- Drag signals to the waveform viewer
- Use zoom and measurement tools to verify timing and logic values
- Confirm outputs match expected results from truth tables

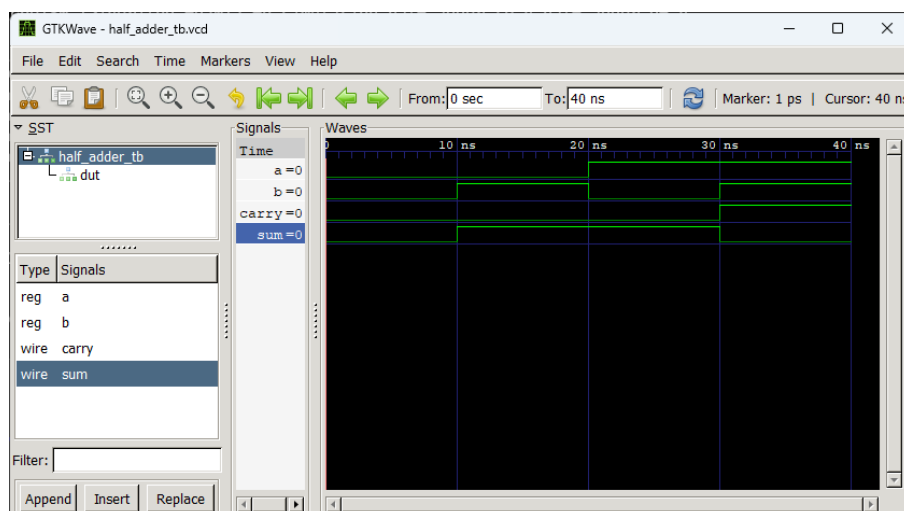


Figure 2.5: Example GTKWave output for a half-adder. Verify that the outputs `sum` and `carry` are produced correctly from the inputs `a` and `b`.

2.5 Lab Tasks and Procedure

1. **Implement the Arithmetic Circuits:** For each of the circuits listed below, create SystemVerilog modules using all three modeling styles (Dataflow, Behavioral, and Structural).

- Half Adder (*Use the examples provided*)
- Full Adder
- Half Subtractor
- Full Subtractor

Notable Information

For the structural Full Adder, a classic approach is to build it using two Half Adders and an OR gate. This is an excellent exercise in hierarchical design. Remember to use SystemVerilog syntax with `logic` data types and modern port declarations. You may also implement these using traditional Verilog syntax for comparison.

2. **Write a Testbench for Each Circuit:** Create a separate testbench file for each of the four circuits (e.g., `full_adder_tb.v`). Each testbench should:

- Instantiate one version of your design (e.g., the dataflow model).
- Apply test vectors that cover every possible input combination from the circuit's truth table.
- Use `$monitor` to print inputs and outputs to the console.
- Generate a VCD file for waveform analysis in GTKWave.
- Use SystemVerilog features like `logic` data types and modern syntax.

3. **Simulate and Verify:** Use Icarus Verilog (with the `-g2012` flag) and GTKWave to compile and run your simulations. For each circuit:

- Confirm that the console output from `$monitor` matches the truth table.
- Analyze the waveform in GTKWave to visually confirm that the outputs change correctly in response to the inputs.

2.6 Challenge Problem: Full Adder-Subtractor Circuit

Design a configurable 4-bit arithmetic unit that can perform both addition and subtraction based on a control signal.

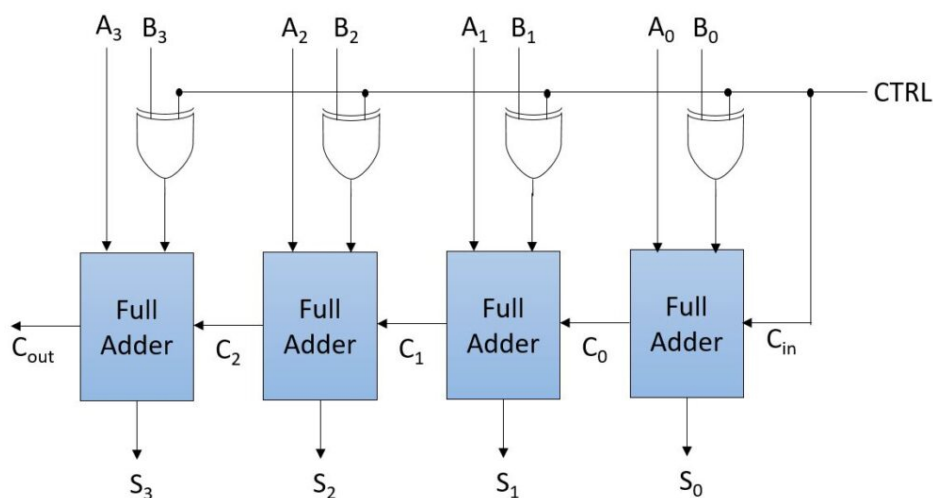


Figure 2.6: 4-bit Adder-Subtractor architecture

Specification:

- **Inputs:** A[3:0], B[3:0], Cin/Bin, Control
- **Outputs:** Result[3:0], Cout/Bout
- **Function:**
 - When Control = 0: Result = A + B + Cin
 - When Control = 1: Result = A - B - Bin

Important Note for Students

The adder-subtractor circuit uses the mathematical identity:

$$A - B = A + (\neg B) + 1$$

When the Control signal is 1 (subtraction mode), the circuit:

1. Inverts each bit of B using XOR gates ($B \oplus 1 = \neg B$)
2. Sets the carry-in to 1 effectively adding 1
3. Performs addition: $A + (\neg B) + 1$ which equals $A - B$

This elegant transformation allows reuse of adder hardware for subtraction operations.

Implementation Strategies:

- Use multiplexers to select between addition and subtraction modes
- Implement using structural design with full adder components
- Create dataflow and behavioral models for comparison
- Develop a comprehensive testbench verifying both operations
- Use SystemVerilog features like packed arrays (`logic [3:0]`) and modern syntax

Important Note for Students

Propagation Delay Consideration: While we simulate with zero delay for simplicity, real hardware exhibits propagation delays. In ripple-carry adders, the carry signal must propagate through each full adder stage, creating a cumulative delay. For wide adders (e.g., 32 or 64-bit), this ripple-carry delay becomes a significant performance bottleneck, motivating more advanced architectures like carry-lookahead adders.

2.7 Pre-lab Questions

1. What distinguishes combinational circuits from sequential circuits?
2. Derive the sum and carry expressions for half and full adders from their truth tables.
3. Derive the difference and borrow expressions for half and full subtractors.
4. Explain the advantages of SystemVerilog's `logic` data type over traditional Verilog's `reg` and `wire`.
5. What is the purpose of the `-g2012` flag in Icarus Verilog?
6. What is the purpose of the `$dumpvars` system task in Verilog testbenches?
7. What are the benefits of using named port connections (`.port(name)`) over positional connections?

2.8 Post-lab Questions

1. Compare and contrast the three SystemVerilog modeling styles used in this lab.
2. Explain the difference between continuous assignments and procedural assignments.
3. Describe the function of process statements and block statements in SystemVerilog.
4. What are sequential statements, and how do they differ from concurrent statements?
5. Explain the difference between high-impedance (Z) and unknown (X) signal states.
6. How does the adder-subtractor circuit implement subtraction using addition?

7. What are the advantages of using `always_comb` instead of `always @(*)`?
8. How does SystemVerilog improve upon traditional Verilog for digital design?
9. Compare the readability and maintainability of traditional Verilog vs. SystemVerilog code.

2.9 Lab Report Requirements

Submit a comprehensive lab report containing:

- Answers to all pre-lab and post-lab questions
- Complete, well-commented SystemVerilog code for all circuits
- Simulation waveforms with detailed analysis
- Discussion of design challenges and solutions
- Comparison of different modeling approaches
- Reflection on the transition from Verilog to SystemVerilog
- Optional: Comparison with traditional Verilog implementations

2.10 Grading Rubric

Grading Criteria	Points
Pre-lab Work	15
Lab Performance	35
Post-lab Work	20
Lab Report	30
Total	100

Lab Report Breakdown:

SystemVerilog code for all experiments	15 points
Output waveforms and analysis	15 points